RAI: Pause-Free Committee Changes and Fork Recovery for Consensusless Payments

Anonymous Author(s)
Regular paper

***Abstract*—FastPay and Nano confirm each account's payments separately, without a global order. When the validators change, old votes can still complete a confirmation after the handover, so deployed systems pause the fast path at every epoch boundary. And when an owner signs two conflicting payments, the account stays locked until the next epoch, or forever. RAI changes validators without pausing. Each epoch closes with a checkpoint built from signed reports of the outgoing validators while the next epoch already votes, under rules that keep the two epochs consistent. The same checkpoint keeps one side of every double spend, which becomes final within two epochs of normal operation, so the owner gets the account back. We prove safety and termination and model-check the handover rules. On a wide-area deployment, a prototype keeps confirming through transitions that stall a stop-and-drain handover, with 6.7 to 38 times lower 95th-percentile latency.**

***Index Terms*—Byzantine fault tolerance, reconfiguration, asset transfer, consensusless payments, equivocation, model checking**

I. Introduction

A payment moves money from one account to another. If every account has a single owner, there is no need to put all payments of the world in one sequence: the owner orders the payments of their own account, and the validators only have to make sure that no two incompatible versions of that account's history both become final. Guerraoui et al. showed that this makes asset transfer strictly easier than consensus [1], and systems such as FastPay [2], Astro [3], Zef [4], and Nano [5] use it to confirm a payment in one or two rounds of votes, with no leader and no block interval.

Such systems leave two problems open, and the two are connected.

Changing the validators. Validators join, leave, and move weight. In Nano the voting weight of a validator is whatever the accounts delegating to it hold right now, read by every node from its own ledger, so two honest nodes can count the same vote with different weight and can disagree on who is in the quorum. FastPay fixes the set of validators for the lifetime of the system and lists reconfiguration as future work, because there is "no natural point" at which all authorities switch [2]. The textbook remedy is to stop: end voting, wait until every pending confirmation has settled, agree on a snapshot, and start the new validators from it. Every transition then pauses the system for as long as the slowest confirmation takes.

Accounts that contradict themselves. An owner who signs two conflicting payments for the same position of their account, by a wallet bug or on purpose, splits the validators' votes. If neither side reaches the quorum, no payment at that position can ever be confirmed. In FastPay such an account "may become inaccessible" [2]; in Nano a fork whose final votes split stays unresolved, and every later payment of that account is blocked behind it (we measure this in Section X). A double spend should cost the attacker, not the honest owner who made a mistake, and it should not make the rest of the system slower: the owner should be able to recover the account once they stop contradicting themselves.

Why one mechanism solves both. A handover that stops can forget a contested position: once every confirmation has settled, no payment there can still become final, so both can be dropped and the owner can start again, which is what Sui and Mysticeti do at the end of each epoch [6], [7]. A handover that does not stop cannot do this, because votes the old epoch already released may still complete a certificate for one side. The checkpoint must keep that side under a lock, and RAI keeps a side even when neither can be final yet, since a position left open can be split again in the next epoch. Resolving forks is thus the price of not stopping, and paying it gives the owner the account back.

RAI. This paper presents RAI, a set of rules that gives an account-local payment system epochs, checkpoints, and committee changes without stopping, and that resolves every fork, so the owner of a forked account gets it back. Time is cut into epochs. In each epoch a fixed committee of validators votes on payments exactly as before. At the end of an epoch every outgoing validator freezes one signed *report* of what it saw; the outgoing committee agrees on one *checkpoint* built from these reports by a fixed recipe; and the next committee, known two epochs ahead, installs the checkpoint without voting on it. The checkpoint settles what the old epoch could still have confirmed and keeps at most one side of every unresolved fork, with a note of what may still happen to it. From then on every validator votes only the kept side, which becomes final within two epochs of normal operation, and the owner continues from it.

The hard part is the overlap. RAI does not stop at an epoch boundary: validators start voting in epoch $e+1$ while epoch e is still closing. Votes already given in epoch e can still complete a confirmation after the boundary, and a checkpoint that ignored such a *latent* confirmation could let the next

epoch confirm a conflicting payment. Four rules keep the overlap safe (Section VII): new-epoch confirmations wait for the checkpoint unless they carry proof that the old epoch cannot contradict them; outgoing validators keep adding that proof for a short while; votes cast before their signer knows the checkpoint do not count toward the fast path; and a validator does not switch sides across the boundary. TLA$^+$ model checking finds a violation in 9 to 23 steps when the overlap rule, late notarization, or the early-vote rule is weakened, and none with the full rule set within the explored bounds (Section IX).

Contributions. (1) A protocol for pipelined epoch transitions in account-local payment systems: immutable signed reports, a deterministic checkpoint recipe with *locks* that record what may still become final and which epoch they protect, lagged committees, and the four overlap rules. (2) A proof sketch of safety and termination relative to two abstract services, set reconstruction and agreement, whose interfaces RAI fixes but whose mechanics it leaves open; full proofs are in the supplement. (3) Model checking of the handover rules, which finds a counterexample without each rule but the no-switching rule and checks the complete rule set, including the rule that guards a whole account prefix. (4) A prototype in the RsNano implementation of Nano, evaluated across six regions against Nano and against a stop-and-drain transition built from the same code, with forks, with a Byzantine validator that equivocates in every phase, and with a committee change, and on one host under validator restarts and a crashing departing validator.

RAI is for authenticated committees with an externally maintained fault bound; it does not address permissionless membership, and its liveness promise to an owner who keeps contradicting themselves is only that their account cannot hurt anyone else.

II. The Problem, by Example

Six validators $V_1, \ldots, V_6$ confirm payments; at most one of them, say V_6, is faulty. A block is confirmed when four validators have given it their *final vote*. Alice signs block A at position 7 of her account, paying Bob, and publishes it.

What a committee change must preserve. Suppose the committee is about to change, and the validators agree on a snapshot of all accounts to hand over. Validators V_1 to V_4 have each signed a final vote for A, but the network is slow: no single party has seen all four votes, and in particular the validators building the snapshot have seen at most three. If the snapshot leaves position 7 open, the new committee may confirm a conflicting block A' there. Later the four old votes meet, and A is confirmed too, by a perfectly valid certificate of the old committee. Two conflicting payments are final. Nothing in the agreement on the snapshot went wrong; the snapshot was simply built from a view of the votes that was not the only possible one. We call A *latent-final*: its certificate exists in the released signatures, though nobody has assembled it. The same holds for a certificate that outgoing validators who have not yet stopped may still complete after the snapshot.

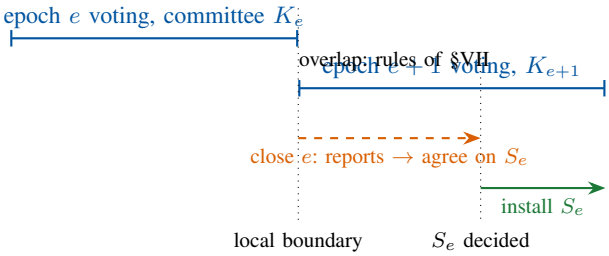


Fig. 1. A pipelined transition. Epoch $e+1$ voting starts at each validator's local boundary while epoch e closes. Most new blocks finalize during the overlap under the overlap rule; the rest finalize when S_e is installed.

The textbook remedy is to stop: end voting, wait until every certificate has either been assembled or become impossible, then snapshot. The wait is as long as the slowest confirmation, at every transition, and during it no payment is confirmed. Section X measures what this costs: under such a pause, blocks published during a transition wait for it.

RAI's answer is that the snapshot does not need to know whether A is final. It needs to know whether A *could* be final, and that is visible in the reports: a correct validator final-votes only a block it has seen notarized, and says so in its report. Four final voters and five reporters out of six overlap in at least three validators, at least two of them correct, so the snapshot sees A notarized in at least two reports and keeps it. The snapshot does not confirm A (it cannot know), but it *locks* position 7 for A, so the new committee cannot confirm A', and A's latent certificate stays compatible with everything that follows.

Accounts that contradict themselves. Now suppose Alice signs two blocks at position 7, A paying Bob and A' paying Carol, and the validators split three against three. No block reaches four, and in Nano the position, and every later block of Alice's, is stuck for good. In RAI the next checkpoint keeps one of the two under a lock, every validator then votes only that block, and it becomes final within two epochs of normal operation, whatever Alice or a faulty validator does (Section V).

III. RAI at a Glance

Figure 1 shows one epoch transition. Three committees are involved: K_e votes on payments in epoch e and, at the end, decides the checkpoint S_e; K_{e+1} is already known (it was fixed by S_{e-1}) and votes in epoch $e+1$ while K_e is still closing; S_e fixes K_{e+2}. Boundaries are local: each validator ends epoch e on its own clock, freezes its report, and opens $e+1$ at once. Nothing requires the validators to stop together.

Three invariants. RAI's rules keep three invariants. *I1 (closure):* every block an epoch may still finalize is held by the checkpoint that closes it, and no block finalized during the overlap contradicts that checkpoint. *I2 (exposure):* every vote that can count toward a certificate reaches the next close, in a frozen report or as evidence its signer holds and reports. *I3 (decision):* once a checkpoint keeps a block at a position,

TABLE I

EACH RULE, THE INVARIANT IT SERVES, AND THE TLA+ COUNTEREXAMPLE WITHOUT IT (LENGTH IN STEPS). WITH ALL RULES, EVERY EXHAUSTIVE RUN PASSES (0.04 TO 121 M DISTINCT STATES), AND THE RECIPE PASSES EVERY RUN TO 17 STEPS (SECTION IX).

Rule	Inv.	Without it
R1 one first vote per position and epoch	I1	assumed by every model
R2 notarize before the final vote	I1, I2	assumed by every model
R3 the second set lists fresh votes for kept blocks	I2	Thm. 1 fails (19); two fast certificates (23)
R4 recovery locks from the highest position down, on settled votes	I1	Thm. 1 fails (15); both sides final (40)
R4, R7 a lock is released only by evidence of its own origin	I1	verdicts differ before and after installation (15)
R4, R5, R10 keep a side; vote only the kept block; no early votes at a contested position	I3	fork open after two good epochs (sim.)
R5 extend a recovery-locked block only once it is notarized again	I1	conflicting finality (16)
R7 the overlap check covers the whole unresolved prefix	I1	conflicting finality (13)
R7 each child needs its own epoch-e witness	I1	conflicting finality (12–14)
R8 late supports count as evidence	I2	the next close cannot verify a correct report (14)
R8 late supports never notarize	I2	a correct report becomes unusable (9)
R9 early votes stay off the fast path	I2	Thm. 1 fails (22, 23)
R10 no switching sides across the boundary	I1	no violation without it (16, 22 M states)
Durable notarization across restarts	I2	both sides final (13, 19)

correct validators vote only for that block there. I1 is safety across the boundary (Theorem 1, Lemma 8); I2 is what lets any $N - f$ reports check I1 (Lemmas 4 and 5); I3 is termination (Theorem 3). Table I lists each rule with the invariant it serves and what model checking finds without it.

The rules, informally, with the thresholds of the six-validator example in parentheses:

Inside an epoch (Section V).

R1 A validator casts at most one *first vote* per account position per epoch, for the first valid block it sees there, and never switches.

R2 A block with q (4) first votes is *notarized*. A validator casts a *final vote* only for the block it first-voted, once it holds that block's notarization. A block is *final* with q (4) final votes, or on the fast path with $N - p$ (5) first votes from validators that know the previous checkpoint.

At the boundary (Section VI).

R3 At its local boundary a validator stops voting in epoch e, signs one immutable *report* naming what it holds final or notarized and what else it first-voted, and opens epoch $e + 1$.

R4 The outgoing committee agrees on a checkpoint built from $N - f$ (5) reports by a fixed recipe that keeps whatever may be final under a *lock*, keeps one side of every fork, and carries older locks until evidence of their own origin epoch releases them.

TABLE II

VOTE THRESHOLDS, AND THEIR VALUES FOR $f = p = 1$, $N = 6$.

What	Needs	Example
Block notarized (also: exclusion witness)	$q = N - f - p$ first votes or supports	4
Block final, normal path	q final votes	4
Block final, fast path	$N - p$ settled first votes	5
Recovery lock in a checkpoint	$r = f + p + 1$ reporter first votes	3
Reports in a checkpoint	$N - f$	5

R5 The next committee installs the decided checkpoint without voting on it. From then on a validator votes at a kept position only the kept block, and builds on a block kept only for recovery once it is notarized again.

While the previous epoch closes (Section VII).

R6 *Gate.* A block of epoch $e+1$ does not become final before S_e is installed, except under R7.

R7 *Overlap rule.* It may become final early if it, and every unresolved block below it, has q (4) supporters in epoch e and q (4) first votes in epoch $e + 1$, and if every older recovery lock it bypasses is released by a witness of that lock's own origin.

R8 *Late notarization.* A validator that has left epoch e but not yet installed S_e also signs an epoch-e *support* for each block it first-votes in $e + 1$, unless it supported a rival at that position in e. They count only toward R7.

R9 *Early votes.* A first vote cast before its signer installed S_e counts toward notarization and toward R7, never toward the fast path. An early final vote is released only with the witnesses of R7 in hand.

R10 *No switching across the boundary.* Until S_e is installed, a validator does not first-vote in $e + 1$ a block that conflicts with one it supported at that position in e, nor any block where it has seen two or a checkpoint keeps one.

Two services are left abstract: how a validator obtains the sets behind another's report (*set reconstruction*) and how the outgoing committee picks one eligible checkpoint (*agreement*). RAI only states what it needs from them (Section IV).

IV. MODEL AND ASSUMPTIONS

Participants. Clients own accounts and sign blocks. A committee has $N = 3f + 2p + 1$ validators of equal weight, of which at most f are Byzantine; p is the number of responses the fast path may do without, not an extra fault allowance. Corruption is static: a Byzantine identity stays Byzantine, a correct one stays correct, including after it leaves. Who may join a committee, and that no committee exceeds f faults, are deployment assumptions. Our running example and prototype use $f = p = 1$, $N = 6$; Table II lists the thresholds that follow.

The numbers are chosen so that any two groups of q validators share at least $f + 1$ members, hence at least one correct one ($2q - N = f + 1$); so that any q final voters

and any $N - f$ reporters share at least $p + 1$ correct ones ($q - 2f = p + 1$); and so that any $N - p$ fast voters and any $N - f$ reporters share at least r correct ones ($N - p - 2f = r$). These three overlaps carry every safety argument of the paper.

Accounts and blocks. Each account is a chain of owner-signed blocks starting at a genesis. A block names its parent and sits at the parent's position plus one; positions are heights in the account, not times. Two blocks of one account *conflict* if neither is an ancestor of the other. A block carries payments: a *send* debits its own account; a *receive* credits the account and names a send that must already be final. An owner *equivocates* by signing conflicting blocks. Safety covers equivocating owners; liveness promises them nothing while they keep contradicting themselves.

Network and cryptography. Signatures cannot be forged and hashes do not collide. Every vote binds its epoch, committee, account, position, kind, block, and the evidence under which it was cast, so a vote cannot be replayed in another context. Correct validators write what they sign to disk before sending it, keep blocks, votes, and evidence, and keep serving them after they leave an epoch. Safety needs no bound on message delay. Termination needs the usual: finite work, fair retry, and recurring windows in which messages arrive within a known delay and clocks drift by a bounded factor.

What a checkpoint contains. A checkpoint S_e is the agreed state at the end of epoch e: for each account the finalized prefix, the unresolved blocks the epoch kept, and *locks* on those blocks; plus balances and the identifiers of received sends. A lock says "keep this block, it may still be final" and records its *strength* (notarized, or recovery) and its *origin*: the epoch whose possible confirmation it protects. A lock is released only by evidence from its own origin epoch (Section VI). The committee of epoch $e + 2$ is a deterministic function of S_e, so it is known throughout the transition from $e + 1$ to $e + 2$.

Two abstract services. RAI commits to sets by their hash and to candidates by their digest, and uses two services to turn these commitments into shared state. *Set reconstruction* lets any validator obtain the two sets a reporter committed to in its signed report; RAI needs that a reconstructed set matches the signed hash (integrity) and that a correct reporter's sets are eventually reconstructible at every correct validator (termination). *Agreement* lets the outgoing committee decide one candidate checkpoint among those RAI declares *eligible*; RAI needs that all correct validators decide the same value, that the decided value is eligible, that the decision proof is verifiable by anyone who knows the committee and cannot be replayed, and that a decision is reached whenever $N - f$ correct members hold an eligible candidate. Any set-reconciliation mechanism and any Byzantine agreement protocol with external validity [8] fit; our prototype uses rateless set reconciliation [9] and a Kudzu-style election [10]. The supplement states the interfaces formally (C1–C2, A1–A4).

Keys of departed validators. Corruption is static even after a validator leaves, and RAI relies on this more than a handover that stops: old-epoch evidence stays admissible after the handover. If $f + 1$ correct members of an old committee later leaked their keys, an attacker could, with the faulty members, sign an epoch-o exclusion witness for a rival of a block that may be final, and release its lock of origin o. A correct validator needs its epoch-o key only until it has installed S_o and stored its epoch-o signatures, so it can sign with a key per epoch, registered in the state that derives its committee, and erase that key then. Released signatures stay verifiable; new ones cannot be made. This needs the prototype to store signatures instead of signing recorded votes again after a restart.

Seats and stake. The prototype gives each of the N heaviest representatives one equal vote, so f bounds faulty seats, not faulty stake. Every safety argument uses only the three quorum intersections above, so we expect them to carry over to stake-weighted quorums; the models do not check this.

Forks as load. Until its position is final, which takes two good epochs, each fork costs a few entries per report, one lock in the checkpoint, and one instance. Load therefore grows with the forks per epoch, not with their history. RAI does not price it; a deployment relies on its base system's anti-spam measure, such as Nano's proof of work [5].

Joining and leaving. The committee of epoch $e + 2$ follows from S_e, so a new member has about one epoch to obtain the ledger before it votes; in our committee change it ran as a standby from the start. A departing member serves blocks, votes, and reports until successor members holding $N - f$ of the weight acknowledge the checkpoint as durable.

Garbage collection. Reports of the next three epochs may cite an epoch's late finality, so the prototype keeps vote records for four epochs and, beyond that, only supports that can still release a surviving recovery lock of their origin. Dropping the rest cannot release a lock, since only evidence of its own origin does, and a kept block becomes final within two good epochs, after which it needs no lock.

V. Inside an Epoch

Within an epoch RAI votes like Nano or FastPay, with two twists that pay off at the boundary: a validator never switches sides, and a final vote is preceded by a notarization.

Confirming a block. Alice signs block A at position 7 of her account and sends it to the six validators. Each validator checks the block against Alice's chain as it knows it (parent, balance, no duplicate receive) and, if it has not voted at position 7 in this epoch, casts a first vote for A (R1). With four first votes A is *notarized*; a validator that holds the notarization and first-voted A casts its final vote. Four final votes make A final. If five first votes arrive from validators that have installed the previous checkpoint, A is final at once, in one round trip; Nano always needs a second round (Section X).

Why the thresholds are safe. Any two groups of four validators out of six share at least two, and at most one of

those is Byzantine. A correct validator supports one block per position (R1), so two conflicting blocks cannot both reach four supporters. Four supporters for A therefore *exclude* every rival: no rival can be notarized or final in this epoch. We call a set of q supporters an *exclusion witness*; it is the one piece of evidence the boundary rules will ask for again and again. Five first votes for A leave at most one correct validator free to support a rival, so a fast-final block has no rival with three honest supporters either; this is what makes the recovery threshold $r = 3$ of R4 work.

A double spend. Suppose Alice also signs A' at position 7, paying someone else, and shows A to three validators and A' to the other three (Figure 2). Neither reaches four. In Nano this position is stuck: final votes cannot be withdrawn, no block at position 7 will ever confirm, and every later block of Alice's account waits behind it. In RAI the next checkpoint decides what may still happen at position 7 (R4). It is built from five of the six reports, which show three first votes for one side and two for the other, say three for A. A is kept under a *recovery lock* of origin e and A' is dropped: from now on no validator admits A' or any other rival of A at position 7, so A' can never become final. Had the reports shown three against three, the block with the smaller hash would be kept.

How the fork ends. The checkpoint decides the position (R5). A validator that has installed it votes only A at position 7, and a validator that has seen both A and A' casts no vote there before it has the checkpoint (R10). So A collects four first and four final votes and becomes final by the second epoch of normal operation after the fork, whatever Alice or the Byzantine validator does (Theorem 3), and Alice's account continues from it. She need not act; to pay sooner she can sign a block C on top of A, admitted once A has four first votes again in the new epoch, and finalizing C finalizes A (Figure 2). A block on top of A' is refused.

Why the checkpoint must decide. Suppose validators could still vote A' whenever they held four epoch-e supporters for it, the witness that releases A's lock; that takes a Byzantine validator that supported both sides. Then in every epoch some correct validators vote A and some A', the Byzantine validator withholds the one vote either side needs, and the position stays open forever, as in Nano (supplement). With the checkpoint deciding, the witness still lets a validator accept a certificate for A' that already exists, so nothing final is contradicted, but it earns A' no new vote.

Durable votes. A correct validator writes its first vote and its final vote to disk before releasing them, and with a final vote the first votes that notarize its block, so that a crash and restart can neither make it vote twice nor lose the notarization its report must show. Our prototype syncs these writes; Section X reports what that costs.

VI. Closing an Epoch

An epoch closes in three steps: every outgoing validator freezes a report, the outgoing committee agrees on a checkpoint built from $N - f$ reports, and the next committee installs it. Figure 3 follows Alice's account through one close.

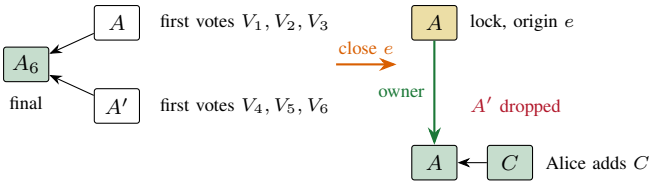


Fig. 2. A double spend at position 7. Neither block reaches four first votes. The five selected reports show three votes for A and two for A': the checkpoint locks A for recovery and drops A', which can never become final. In the next epochs every validator votes A, which becomes final; Alice may also add C on top of A.

report V_1	A_6: F	A: N	B: voted
report V_2	A_6: F	A: N	B: voted
report V_3	A_6: F	A: N	B': voted
report V_4	A_6: F	A: N	B: voted
report V_6	A_6: F	A': voted	

checkpoint S_e: A_6 final (inherited); *A locked, notarized* (V_1 to V_4 show it notarized; A', which only V_6 voted, conflicts with A and is dropped);
B locked for recovery, origin e (three reporter first votes, no rival with three); B' dropped.
Promotion: A is the only block kept at position 7 and has an epoch-e notarization $\Rightarrow$ A final in S_e.

Fig. 3. Five of six reports close epoch e for Alice's account. F = final, N = notarized, voted = first-voted but not notarized (the report's second set).

Reports (R3). When a validator's local clock says the epoch is over, it stops first- and final-voting in it, and signs one header committing to two sets. The first is a tagged view of its ledger: every block it holds, tagged *final* (by an inherited checkpoint or by a certificate it has seen), *notarized* (four first votes seen, no finality), or *inherited* (kept by the previous checkpoint under a recovery lock, nothing new seen). The second set lists every block the validator first-voted in this epoch that the first set does not already show as notarized or final. The two sets are frozen: later events change the validator's live view, never its report. The header carries only the hashes of the two sets; whoever needs the sets reconstructs them (Section IV). A report is *usable* at a validator once both sets are reconstructed and every claim in them is backed by evidence that validator holds: four first votes behind every "notarized", a certificate behind every "final", the reporter's own signed first vote behind every entry of the second set. A Byzantine reporter can make its own report unusable, and nobody else's.

The second set has a subtle requirement. A reporter that first-voted a block the previous checkpoint already keeps under a recovery lock tags it *inherited* in the first set. If the second set excluded every block of the first set, this fresh vote would appear nowhere, and a latent fast certificate could vanish from

the checkpoint (Section IX). The second set therefore excludes only blocks tagged notarized or final.

The recipe (R4). A candidate checkpoint names exactly $N - f$ usable reports from distinct members, in a fixed order, plus a manifest of the evidence behind their claims. From these inputs every validator computes the same state by the following recipe, which is deterministic and never consults the candidate's own certificate.

1) *Keep finality.* Every block a selected report shows final, with its ancestors and the sends its receives depend on, is final.
2) *Lock notarized blocks.* At each position, if a selected report shows a block notarized, backed by four first votes, keep it under a *notarization lock*. Two conflicting blocks cannot both be notarized (Section V), so there is at most one.
3) *Lock for recovery.* Count only *settled* first votes, cast after installing S_{e-1}. From the highest position down, where no block is notarized or kept by a lock placed above, lock a block with r (3) of them under a *recovery lock* of origin e. Then, from the lowest up, where nothing is kept, keep the previous checkpoint's block if eligible, else the one with the most, ties by hash. A block that may still become fast-final has r, so it or a child of it is locked; keeping a side even below three lets the next epoch converge.
4) *Carry old locks.* Every lock of the previous checkpoint is carried forward, unless a final block excludes its block, or, for a recovery lock of origin o, the manifest contains an exclusion witness of epoch o for a conflicting block. Nothing else releases a lock; in particular a newer epoch number never does.
5) *Promote.* For each account, starting at its last final block, walk up while the next block is the only one kept at its position and has an epoch-e notarization of first votes; those blocks become final with the checkpoint. Promotion never resolves a fork and never promotes a block kept by a recovery lock alone.

The thresholds make this safe (Theorem 1): whatever the old epoch could still confirm is kept as final or under a lock, and a lock can only be released by evidence of its own origin epoch, which cannot exist if the block it protects can still be final.

Why locks have an origin. A recovery lock says "three validators first-voted this block in epoch o; a fast certificate with five epoch-o votes may exist." Such a certificate is excluded exactly by four epoch-o supporters for a rival, and by nothing else: four epoch-$(o+1)$ votes for the rival say nothing about epoch o. So the lock records o, and a block may carry locks of several origins, one per epoch in which it was voted but not resolved. Releasing a lock on the strength of a newer epoch's votes would let two validators finalize different blocks (Section IX).

Eligibility, agreement, and installation (R5). A candidate is *eligible* if its $N - f$ reports are usable, its manifest hash matches, and the recipe yields the state whose digest it names. The outgoing committee runs the agreement service with this predicate; the service may be any Byzantine agreement protocol with external validity. The decided value is small, $N - f$ report headers and two digests, whatever the number of accounts. Since each header carries its reporter's signature, a validator that holds another version of a report, which only a Byzantine reporter can sign, authenticates the selected one from the value and reconstructs it. The next committee verifies the decision proof, reconstructs the inputs, recomputes the state, and installs it: its final set is the union of its base and what it had already finalized, replay applies each operation once, and nothing is rolled back. There is no second vote; the successor only checks.

After the checkpoint: the checkpoint decides. A lock guarantees that no rival of the kept block is admitted at its position until evidence of the lock's own origin releases it; a notarization lock is never released. The checkpoint also decides how validators vote there (R5): a kept block gets an instance at its own position in the open epoch, and a validator that has installed the checkpoint first- and final-votes only that block there, and above it only on top of it, even if it holds a witness that would release the lock. The witness still lets it accept a rival's certificate that already exists (Lemma 8), so this only withholds votes. A validator that has seen two blocks at a position also waits for the checkpoint before voting there (R10). Together with a recipe that always keeps a side, this makes the kept block final within two epochs of normal operation, with or without the owner (Theorem 3). The owner may also extend the kept block with a fresh child: at once under a notarization lock, and under a recovery lock once the block is notarized again in the open epoch. Letting any kept tip be extended at once would be unsafe: a dropped side with an old witness may become final in the new epoch while a child's certificate finalizes the kept side (supplement, Case 5). In our fork workloads, whose owners never extend a forked position, validators resolved every fork this way (Section X).

VII. Voting While the Previous Epoch Closes

A validator opens epoch $e + 1$ the moment it leaves epoch e, typically a few seconds before S_e is decided. During this overlap two things could go wrong: the new epoch could confirm a block that the old epoch's latent votes contradict, and the checkpoint could later rule against something the new epoch already confirmed. Rules R6 to R10 prevent both. We explain them with Alice's account at position 8, right after A became final at position 7.

The gate (R6). The simplest safe behaviour is to wait: a block of epoch $e + 1$ collects first and final votes as usual, but no validator treats it as final until it has installed S_e and re-checked the block against it. If Alice's block B at position 8 is published during the overlap, its certificate is held, and the moment S_e arrives the held certificate takes effect, or is discarded if S_e kept a conflicting block. This is correct but it is also the stop-and-drain cost in disguise: every block published during a close waits for the close to end.

The overlap rule (R7). A block can skip the gate if the old epoch provably cannot contradict it. The proof is an exclusion

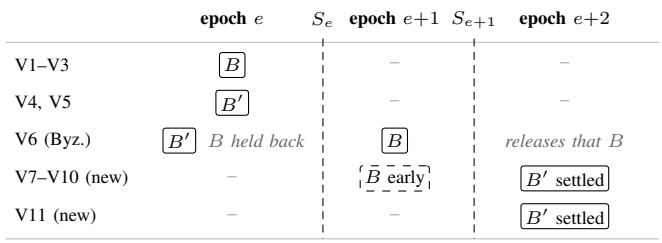
witness in the closing epoch: four supporters for B in epoch e. If four validators supported B at position 8 in epoch e, no rival at position 8 can be notarized or final in epoch e (Section V), so S_e cannot finalize a rival, cannot lock a rival for notarization, and any recovery lock S_e places on a rival has origin e and is released by exactly this witness. So a validator that holds the epoch-e witness for B, four epoch-$(e+1)$ first votes for B, and an epoch-$(e+1)$ final certificate for B may finalize B before S_e is known. The same check must hold for every unresolved block below B on Alice's chain, because finalizing B also finalizes its ancestors; and if B bypasses a recovery lock inherited from an older checkpoint, the validator must hold a witness of that lock's own origin epoch. The rule is exactly the check a validator would perform after installing S_e, so a validator that finalizes B before and one that re-checks after reach the same verdict (Lemma 8).

Late notarization (R8). A block published after the boundary has no epoch-e votes at all, so the overlap rule would never apply to it and it would wait at the gate. Late notarization fixes this. A validator that has left epoch e but not yet installed S_e, when it first-votes B in epoch $e+1$, also signs an epoch-e *support* for B, unless it supported a different block at position 8 in epoch e. These late supports count toward the exclusion witness and toward nothing else: no notarization, no final vote, no report entry rests on them, since reports are frozen before they are cast. Safety is the same argument as before: one validator supports one block per position in epoch e, by a first vote or by a late support.

Early votes (R9). A fast certificate must not rest on votes whose eligibility no correct signer can show. Fig. 4 gives the run. In epoch e, V1–V3 vote B and V4–V6 vote a rival B'; the Byzantine V6 also signs a vote for B and holds it back, and S_e keeps B' under a recovery lock. Before installing S_e, four new members first-vote B in epoch $e+1$; with V6 that makes five. Once V6 releases its epoch-e vote, $\{\text{V1}, \text{V2}, \text{V3}, \text{V6}\}$ is an exclusion witness that releases the lock, so the five votes would be a fast certificate for B. No correct validator held that witness, however, so S_{e+1} keeps B', which epoch $e+2$ finalizes. The rule removes the dependency: a first vote names the checkpoint its signer had installed; early votes, cast before S_e, count toward notarization and witnesses but never toward a fast certificate, and an early final vote is released only by a validator that holds the overlap witnesses, which its report then carries to the next checkpoint. A block first-voted during a close therefore takes two rounds instead of one.

No switching (R10). A validator that supported B' at position 8 in epoch e does not first-vote a conflicting B in epoch $e+1$ until it has installed S_e. Otherwise it could contribute to an epoch-$(e+1)$ notarization of B while its epoch-e vote still helps B' toward a latent certificate. At a position where it has seen two blocks, or which a checkpoint keeps, it casts no first vote at all before installing S_e: re-voting its old side early would let the split repeat in every epoch.

After installation. When S_e arrives, each validator re-checks its provisional epoch-$(e+1)$ work against the finalized history, the frontier, and the locks of S_e. Held certificates take effect; work on a block that S_e excluded is discarded, without clearing the validator's recorded first vote. Every position with one kept block gets an instance at that position in the open epoch, and a rival is refused while any lock on the kept block stands. If a kept block does not finalize in epoch $e+1$ either, its first votes are exposed again in the next reports, and the next checkpoint keeps it again, with a lock of origin $e+1$ if three reporters voted it: a block may carry locks of several origins, each released only by a witness of its own epoch. From then on every validator that installed the checkpoint votes it there.



S_e: reports V2–V6; B' has 3 settled votes, so it is kept under a recovery lock of origin e.
S_{e+1}: no correct member holds the witness $\{\text{V1}, \text{V2}, \text{V3}, \text{V6}\}$ for B, so B' stays kept.
Epoch e+1: $\{\text{V6}, \ldots, \text{V10}\}$ is a fast certificate for B if early votes count.
Epoch e+2: $\{\text{V7}, \ldots, \text{V11}\}$ is a fast certificate for B'.

Fig. 4. Why early votes stay off the fast path (R9). Without the rule, B and B' both become final once V6 releases its held-back vote. Dashed box: a first vote cast before its signer installed S_e.

VIII. Why It Is Safe and Why It Terminates

We state the results and give the reason for each in a sentence or two; the supplement has the full proofs, numbered as here. Safety relies on the integrity of set reconstruction and on the agreement, eligibility, and verifiability of agreement; termination also relies on their termination properties.

Three lemmas carry the arguments. *Account exclusion* (Lemma 1): two sets of q supporters share a correct validator, so at most one block per position has an exclusion witness, and a block that can become fast-final leaves every rival fewer than r supporters. *Notarized ancestry* (Lemma 2): a child is first-voted only on a parent that is final, under a notarization lock, or notarized in the child's epoch, so two conflicting final branches meet at one position where Lemma 1 or a lock excludes one. *Exposure* (Lemma 4): a correct reporter's frozen report shows every block it final-voted as notarized or final and lists every other block it first-voted, including blocks the predecessor already kept.

Theorem 1 (Closure preserves latent and late finality): If an epoch-e final certificate for B can be formed from released votes, even one completed after the reports froze or after S_e was decided, then every eligible checkpoint S_e holds B: as final, under a notarization lock, or under a recovery lock of origin e. No admissible evidence releases that hold.

Why. A normal certificate's q final voters and the $N-f$ selected reporters share $p+1$ correct validators, each reporting B notarized; a fast certificate's $N-p$ voters share r correct

ones, so B reaches r settled reporter votes while no rival does, and the recipe, working down, locks B or a child of it. Only an epoch-e witness for a rival could release that lock, and Lemma 1 excludes one.

Lemma 8 (Same verdict before and after installation): A validator that finalizes B under the overlap rule before installing S_e accepts B if and only if it would accept B by re-checking after installing S_e; and a later checkpoint verifies a report that claims B final.

Why. The overlap rule asks for exactly the evidence that re-checking consults: the epoch-e witness rules out a final or notarized rival in S_e and releases any origin-e lock on one, and matching-origin witnesses release any older lock.

Theorem 2 (Transfer safety across epochs): Finalized histories at correct validators are compatible and never rolled back, finalized balances are nonnegative, and each send is received at most once.

Why. By induction over epochs, jointly with Theorem 1: inside an epoch Lemmas 1 and 2 apply; at a boundary every correct validator installs the same eligible checkpoint, which preserves every formable old certificate; and in the overlap, Lemma 8 makes all correct validators agree on which blocks are final.

Theorem 3 (Termination): Every started handover is eventually decided and installed by correct validators. An owner whose new blocks are uncontested sees them final and in a checkpoint eventually. A forked position whose parent is final becomes final, on one side, by the end of any two consecutive epochs of normal operation, whatever the owner and the Byzantine validators do; the owner then continues from it.

Why. $N - f$ correct reports become usable, so agreement decides an eligible checkpoint. At a fork still open after a good epoch, every correct validator has seen both blocks; the next checkpoint keeps a side, preferring the one it kept before, and in the next good epoch all of them vote only that side, which becomes final.

IX. Model Checking the Handover

Several rules of Sections VI and VII guard against interleavings that are easy to miss in a written argument. We wrote TLA$^+$ models of the handover and checked them with TLC, each with the full rule set and with switches that weaken one rule, to show that the rule is needed. Each takes one account with $N = 6$, $f = p = 1$, five correct validators and one Byzantine validator whose signatures are available for every block, vote kind, and epoch, and whose report is chosen by the adversary subject to the usability checks. Agreement is the nondeterministic choice of one eligible candidate; TLC explores every interleaving, with validators identified up to permutation. The models, configurations, and TLC outputs with the violating traces are in the supplement and the artifact.

There are eight models. *Close* runs three epochs and two checkpoints with locks of several origins; *Overlap* has validators cross their boundary one by one, vote early, late-notarize, and finalize under the overlap rule; *Handoff* adds a committee change with the Byzantine validator in both committees; *Prefix* exercises the prefix-wide guard of R7 on two positions, with identical, disjoint, or overlapping committees; *Recovery* lets the owner extend a forked block; *Resolve* checks that two good epochs close a fork; *Restart* restarts validators before they freeze their report; and *Recover* runs four epochs and three checkpoints with two or three rivals. Each has switches that weaken one rule.

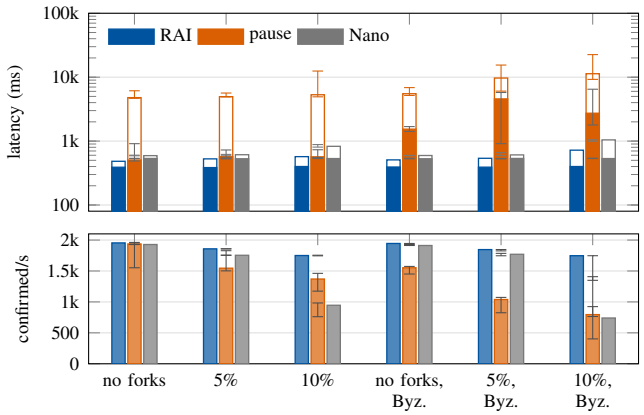


Fig. 5. RAI ($f = p = 1$), pause, and Nano across six regions at 2,000 blocks/s offered: no forks, 5%, and 10%, without and with a Byzantine validator. Top: median (filled) and 95th-percentile (outline) latency of non-fork blocks. Bottom: non-fork blocks confirmed per second over the run. Bars are medians over three runs, whiskers their range.

Table I lists the counterexamples by rule; the supplement lists every run and explains each violation. Extending any kept tip is unsafe even without early votes. A recipe that counts all first votes and works bottom-up loses a fast-final child, and so does a bottom-up recipe on settled votes with a forking owner; the recipe of Section VI passes the safety, side-change, and recovery checks by simulation, and safety in every run of up to 17 steps, two more than the bottom-up order needs to fail. Probes show that the invariants are not vacuous.

The models are small: one or two positions, two to four epochs, no receives, restarts in one model only, $N \in \{4, 6\}$. Their abstractions only add behaviours, except the recover model's owner and Byzantine policies, so a violation in the model is a violation of the protocol, but a clean run is evidence, not proof.

X. Implementation and Evaluation

The evaluation asks three questions. Does RAI resolve the forks Nano leaves open, and what does that cost in confirmation rate and latency? Does it keep confirming payments while an epoch closes, where a stop-and-drain transition stops, also when a validator is Byzantine? Does a committee change disturb it?

Prototype. We implemented RAI in RsNano [11], a Rust implementation of the Nano node [5]. Account elections follow Section V, with late notarization, the early-vote rule, and the prefix-wide overlap rule; checkpoints carry origin-stamped locks and are built by the recipe of Section VI. Epochs end at fixed times $T_0 + kD$ on every validator's local

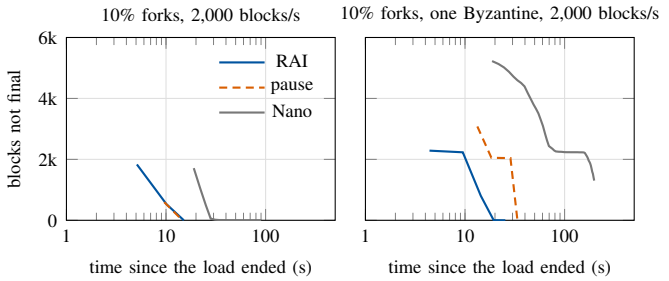


Fig. 6. Blocks not yet final on the validator with the most of them, after the load ended: forked positions and the blocks behind them (one run each, sampled from the first check after the load). RAI and pause finalize all of them, identically on every validator; Nano leaves blocks open.

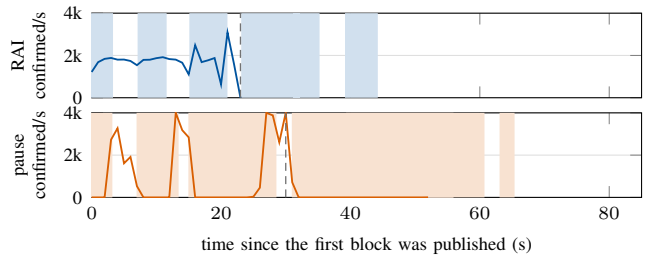


Fig. 7. Non-fork blocks confirmed per second at 2,000 blocks/s offered, with 10% forks and one Byzantine validator, across six regions (one run each). Shaded: an epoch is closing, from the first validator leaving it to the last one seeing its checkpoint decided; dashed: the end of the load. Pause stops confirming during each close, then catches up in bursts (cut off at 4,000/s); RAI keeps confirming through the closes.

clock. The content of every vote, the first votes behind every final vote, and every frozen report, decided checkpoint, and overlap certificate are synced to disk before a signature that depends on them leaves the process, so a restarted validator refuses whatever its earlier signatures forbid, reports what it notarized, and signs its recorded votes again to serve them as evidence. Set reconstruction uses rateless invertible Bloom lookup tables [9]. Agreement is one leader-based election per handover in the style of Kudzu [10]: a validator votes for a proposal only after reconstructing its reports, running the recipe, and matching the state digest; leaders rotate, and a round whose leader stays silent times out after 2 s. The supplement lists where the prototype departs from the protocol.

What we compare. *RAI* is the protocol of this paper with $f = p = 1$ on six members: a certificate needs four of six, and the fast path five settled first votes. *Pause* is the same code with one switch flipped: no instance of epoch $e+1$ starts before S_e is decided locally. Reports, recipe, agreement, and installation are unchanged. *Nano* is the RsNano `develop` node with one optimization of RAI's prototype ported to it, checking a fork block's signature once: a block is confirmed once final votes reach 67% of the online weight, five of six here, after a round of non-final votes. It has no epochs and no committee, and since final votes cannot be withdrawn, a fork whose votes split leaves neither side able to confirm. RAI and Nano both tolerate one Byzantine validator, but RAI's normal path needs fewer votes.

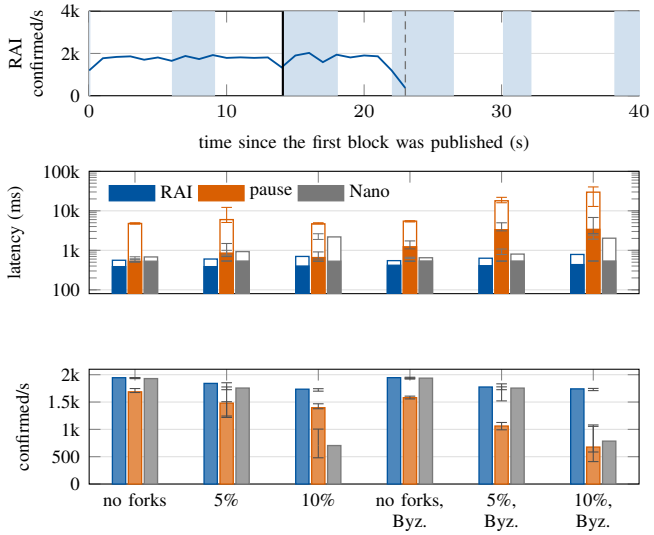


Fig. 8. A committee change under load across seven regions at 2,000 blocks/s: Montréal hands its weight to Toronto, so the committee of epoch 3 has Toronto in its seat. Top: RAI with 10% forks, the solid line marking the first validator entering epoch 3 (one run). Below: latency and confirmation rate as in Figure 5, three runs each, without and with a Byzantine validator that keeps its seat.

Setup. Six equal-weight validators run on separate Google Cloud VMs (n2-standard-4, four vCPUs), one each in Warsaw, Iowa, Montréal, Las Vegas, Taiwan, and Melbourne, with round trips from about 30 to over 300 ms. A load generator on another VM in Warsaw publishes 45,000 blocks at 2,000 blocks/s, each on the frontier of a randomly chosen account, with the same random choices for the three systems. An account publishes its next block only once its last one is confirmed, so a system that confirms slowly is offered less: the client published under 95% of the offered non-fork rate in 20 of pause's 36 runs, down to half of it, and in 4 of RAI's and 2 of Nano's, down to 75 and 85%. In the fork scenarios it publishes a conflicting sibling for 5% or 10% of the blocks, each sibling to half of the validators, and never builds on a forked position, so what is left open measures what the validators resolve on their own. In the Byzantine scenarios one validator runs a modified node: it votes for both blocks of every fork, signs two versions of its report and sends each to half of the validators, and leads the first round of every close with two different proposals. Nano's Byzantine validator votes for both blocks of every fork, the one attack Nano has a phase for. RAI and pause use 8 s epochs, so a close starts about every 8 s during the load. The *confirmation rate* counts the non-fork blocks the client sees confirmed per second over the whole run, so with 10% forks it cannot exceed 1,800. *Latency* runs from a block's first publication to its confirmation at the Warsaw validator and counts non-fork blocks only. After the load, the validators keep running until no block is open on any of them, or until none has become final for 30 s or the harness's time limit is reached. Each cell ran three times; we

report the median and the range. A run passes when every validator holds the same blocks and the decided values agree and, for RAI and pause, every block is final everywhere.

Latency and confirmation rate (Figure 5). All 108 runs passed. Without forks RAI has a median of 390 ms and a 95th percentile of 483 ms against Nano's 535 and 592 ms, and the two confirm about the same (1,954 and 1,928 blocks/s). Nano needs five of six votes in both of its rounds, so it waits for a distant validator twice; RAI's normal path waits for four, and 14 to 69% of its finalizations, depending on the validator and run, take the one-round fast path. With 5% and 10% forks RAI's median stays at 385 and 400 ms and its 95th percentile at 527 and 571 ms, at 1,858 and 1,750 blocks/s. Nano keeps its median (529 and 533 ms) and has a 95th percentile of 612 and 828 ms, but with 10% forks its 99th percentile is 14 to 19 s and it confirms 946 blocks/s, since the run lasts until its last non-fork block is confirmed. With a Byzantine validator RAI's median is 391 to 400 ms and its 95th percentile 508 to 721 ms (994 ms in one run), at 1,747 to 1,945 blocks/s; Nano's are 529 to 535 ms and 595 to 1,044 ms, at 740 to 1,912 blocks/s. Pause is behind both: over its runs the median is 0.5 to 6.5 s and the 95th percentile 4.7 to 23 s. Cell by cell its median is 1.3 to 12 times RAI's and its 95th percentile 9 to 18 times, and RAI confirms up to 2.2 times as many blocks per second.

RAI resolves forks (Figure 6). With forks, RAI finalized every open block within 15 to 21 s of the end of the load in all six runs without a Byzantine validator, and within 22 to 40 s with one. Pause took 14 to 21 s and 22 to 43 s. In all of these runs every validator cemented the same state. RAI's recipe counts settled first votes only, so a fork whose first votes all came before the previous checkpoint was installed waits one more epoch. Nano did not finalize them all. When the harness stopped waiting it had 3 to 915 blocks open on a validator without a Byzantine validator and 13 to 1,303 with one. Across all 24 of its runs with forks, committee change included, its validators held the same blocks, but in 14 of them had cemented different numbers of them.

What a pause costs. A transition that stops confirming for a close of length $c \le D$ once per epoch of length D stalls for a fraction c/D of the time. A block published t into the stall waits an extra $c - t$, so for a uniformly random publication time the extra delay W satisfies $\Pr[W > w] = (c - w)/D$ for $0 \le w \le c$. Its mean is $c^2/(2D)$, and its 95th percentile is $c - 0.05D$ whenever $c > 0.05D$, and zero otherwise. With $D = 8$ s, any close longer than 0.4 s therefore reaches the 95th percentile almost in full, which is why pause's 95th percentile (4.7 to 23 s) follows its close times. Sui's pause of a few hundred milliseconds in epochs of about a day [6], [12] never reaches its 95th percentile. Stopping is cheap only when $D > 20c$, about two minutes for a 5 s close, and with such epochs an account caught in an equivocation waits up to an epoch. RAI does not stall at the boundary, so it can use short epochs, and a fork resolves within two of them.

RAI keeps confirming while an epoch closes (Figure 7). Pause holds new blocks while an epoch closes and then confirms them in a burst. With 10% forks and a Byzantine validator it had 14 to 20 seconds without a single confirmation, up to 13 in a row, and averaged 362 to 659 blocks/s while an epoch closed. RAI, on the same load, had no such second in two runs and eight, up to four in a row, in the third, and averaged 1,626 to 1,738 blocks/s while an epoch closed. Without a Byzantine validator RAI had no such second across nine runs and averaged 1,724 to 1,977 blocks/s during closes; pause had 5 to 8 and averaged 143 to 971. RAI's closes are not short: with 10% forks and a Byzantine validator the longest of each run takes 5.1 to 13.9 s, up to longer than an epoch, since a close must wait for the reports of all five honest validators (pause: 16 to 25 s). But no block waits for them unless the fork rules hold it. Each validator finds at least 12,900 of the 45,000 blocks eligible for the overlap rule.

A committee change (Figure 8). A seventh validator in Toronto starts outside the committee with a small balance, and Montréal moves its whole balance to an account delegating to it during epoch 1, so the committee derived from that epoch, the one epoch 3 runs, has Toronto in Montréal's seat. The load starts 2 s into epoch 1, so epoch 3 begins about 14 s into it. Without a Byzantine validator RAI's median is 386 to 400 ms and its 95th percentile 560 to 700 ms (2.0 and 2.3 s in two runs), at 1,735 to 1,945 blocks/s, close to the cells without the change; its slowest closes take 3.9 to 11.5 s, and it finalizes every open block within 25 s of the end of the load. With a Byzantine validator that keeps its seat, the median is 408 to 435 ms and the 95th percentile 548 to 786 ms (1.2 and 1.6 s in two runs) at 1,741 to 1,946 blocks/s, and every block becomes final within 34 s. Closes are slowest with forks, a Byzantine validator, and the change: the slowest close of each run takes 6.3 to 14.8 s, against 8 s epochs. Pause's median is 0.5 to 3.4 s and its 95th percentile 4.7 to 30 s, 6.7 to 38 times RAI's. Nano has no committee; moving the balance only changes its weights. Every member holds an equal share and the standby a sixty-fourth of one, so its threshold still takes five of six votes: its median is 530 to 537 ms, as in its cells without the change, and it confirms 410 to 1,081 blocks/s with 10% forks.

Restarts and handoff. A second harness, on the same host, kills validators mid-load or as a close starts, restarts them on their data, and checks every logged signature for conflicts. Restarted validators rejoined by checkpoint catch-up, and a departing member killed when its last close starts and kept down 30 s left every checkpoint decided identically. No run produced a conflicting signature. Five further restart runs restored every notarization, and writing them cost no measurable rate or latency. Synced signing costs about 130 ms at the median and nothing in confirmation rate. The supplement reports these runs.

Threats to validity. RAI's normal path needs four votes and Nano's five. At Nano's quorum (six seats, $p = 0$, no fast path; 18 further runs, all settled) RAI's median is 538 to 549 ms against Nano's 529 to 535, and its 95th percentile 739 to 1,126 ms against 592 to 1,044: RAI's lower latency comes from its quorum and fast path. Its rate stays at 1,704 to 1,888

TABLE III
HOW SYSTEMS THAT CHANGE VALIDATORS TREAT THE EPOCH BOUNDARY AND AN OWNER WHO SIGNS CONFLICTING PAYMENTS, AS THE CITED WORK DESCRIBES THEM (—: NOT DISCUSSED THERE).

System	Fast path at handover	Late old-epoch certificate	Equivocating owner
Sui Lutris [13], [6], [12]	paused (∼100s of ms)	retried next epoch	locked to epoch end (∼1 day)
Mysticeti-FPC [7]	paused	in last commit's history	unlocked after the epoch
Carbon [14]	old view fenced	never commits	operations excluded
Linera [15], [16]	per chain, on migration	valid until epoch removed	per-chain rounds
PDCC [17]	continues	only for installed config.	—
Pause (Sec. X)	paused	drained	as RAI
RAI	**continues**	**kept by next checkpoint**	**one side final in 2 epochs**

blocks/s. Latency is measured at one validator, beside the load generator; a client elsewhere would see other numbers. It covers only the blocks the client published, and pause, which confirms slowly, was offered less than the full rate, so its latency is understated. Restarts were not run across regions, and the six-region runs did not audit signatures for conflicts.

XI. RELATED WORK

Payments without consensus. Asset transfer needs no consensus [1]. FastPay settles prefunded payments without a global order [2], Astro and Carbon build payments on reliable broadcast [3], [14], Zef adds private coins [4], and Nano uses account chains with open representative voting [5]. Table III compares how the systems that change validators treat the boundary and an owner who equivocates.

Handovers that pause. Sui Lutris proves the reconfiguration of a consensusless fast path safe; its handover waits for $2f + 1$ validators to end the epoch, drops the lock stores, and rolls back transactions that no checkpoint contains [13]. Sui holds user transactions for a few hundred milliseconds at the boundary and keeps an equivocated object locked until the next epoch, about a day later [6], [12]. Mysticeti-FPC pauses its fast path with an epoch-change bit and unlocks transactions that equivocation blocked once the epoch ends [7]; Cuttlefish broadens the fast path with collective objects and FastUnlock [18]. Their pause is short because consensus already orders every certificate. RAI has no ordering layer, and its close needs reports and an agreement that take seconds, so it overlaps the close instead of pausing; unable to drop both sides of a fork, it keeps one (Section I).

Handovers that fence or migrate. Carbon lets clients vote validators in and out and moves between views as dynamic Byzantine reliable broadcast does; once a new view is approved, operations addressed to the old one no longer commit [14], [19]. Pastro and asynchronous Byzantine reconfiguration treat changing participation more generally [20], [21], and DynaStore, FreeStore, SmartMerge, and reconfigurable lattice agreement change the replicas of a storage service without consensus [22], [23], [24], [25]. PDCC accepts payment certificates only for the installed configuration and assumes correct members common to consecutive configurations [17]. In Linera each chain moves to a new epoch with one of its own blocks, so epochs overlap and an old committee serves until its epoch is removed [15], [16]. RAI keeps every certificate the old epoch can still complete, needs no members in common, and moves all accounts at once, with no owner acting.

State machines and view change. Replicated state machines reconfigure through their log: a configuration chosen by command i governs commands from $i{+}\alpha$ on [26], Stoppable Paxos stops one configuration before the next [27], Vertical Paxos switches through an auxiliary master [28], Raft passes through a joint configuration [29], BFT-SMaRt orders reconfiguration requests [30], and in PaLa both committees vote during a handover [31]. RAI has no log to cut; its lag of two epochs between deciding a committee and using it plays the role of α. Its resilience $N = 3f{+}2p{+}1$ is that of SBFT and Kudzu [32], [10], and its checkpoint is a view change run per account position: a notarization lock re-proposes a prepared block, and a recovery lock keeps one that may have committed on the fast path, found from $r = f + p + 1$ of $N - f$ reports as SBFT's new primary finds it [32]. Unlike a view change, it has no leader, one committee decides it while the next installs it, and the next epoch votes before it completes.

Building blocks. Agreement with external validity is a standard interface [8], instantiated here in the style of Simplex and Kudzu [33], [10]; set reconciliation is mature [34], [35], [9]; and model checking small configurations of Byzantine protocols with TLC is common practice [36].

XII. LIMITATIONS AND CONCLUSION

RAI's checkpoints grow with the unresolved forks they keep, and the garbage-collection argument of Section IV covers only the prototype's four-epoch window. The committee model is authenticated and equal-weight with an externally kept fault bound. A close waits for every honest report, so with forks and a Byzantine validator it can outlast an epoch; the restarts were measured on one host only; each cell ran three times; and process kills test crash consistency, not power loss. The models cover at most two positions and four epochs and check the recipe only by simulation and to 17 steps.

Within these limits RAI changes the validators of account-local payments without stopping: the next epoch votes while the previous one closes, and the checkpoint keeps whatever the old epoch could still confirm. It keeps one side of every contested position and finalizes it within two epochs of normal operation, so the owner gets the account back. Across six and seven regions it keeps confirming through every boundary, with a 95th percentile 6.7 to 38 times lower than a stop-and-drain handover, and it finalizes the forks Nano leaves open.

AI Tool Usage

ChatGPT (OpenAI, model and version) helped structure the manuscript, check sources, review the protocol and proofs, and write the TLA+ models, which the authors checked with TLC. Claude Code (Anthropic; Claude Opus 5.5, Claude Fable 5.1 and 5.2) helped implement and debug the prototype, benchmark harness, and stop-and-drain switch; ran the experiments of Section X, produced Figures 5–8, and drafted the text and Figures 1–3, which the authors revised. Claude (Anthropic, Claude Opus 5.5) drafted the abstract, one paragraph of Section I, the invariants and Table I, the last five paragraphs of Section IV, Fig. 4 with its paragraph, the pause model of Section X, and Section XI with Table III, which the authors checked and revised. The authors verified every reference. The proof arguments were reviewed by the authors and are not machine-checked beyond the finite configurations of Section IX. No data is AI-generated: all measurements come from the prototype runs of Section X.

References

[1] R. Guerraoui, P. Kuznetsov, M. Monti, M. Pavlovic, and D.-A. Seredinschi, "The consensus number of a cryptocurrency," in *Proceedings of the ACM Symposium on Principles of Distributed Computing (PODC)*, 2019, pp. 307–316.

[2] M. Baudet, G. Danezis, and A. Sonnino, "FastPay: High-performance byzantine fault tolerant settlement," in *Proceedings of the ACM Conference on Advances in Financial Technologies (AFT)*, 2020, pp. 163–177.

[3] D. Collins, R. Guerraoui, J. Komatovic, M. Monti, A. Xygkis, M. Pavlovic, P. Kuznetsov, Y.-A. Pignolet, D.-A. Seredinschi, and A. Tonkikh, "Online payments by merely broadcasting messages (extended version)," arXiv:2004.13184, 2020, extended version of the DSN 2020 paper; Appendix A describes asynchronous reconfiguration.

[4] M. Baudet, A. Sonnino, M. Kelkar, and G. Danezis, "Zef: Low-latency, scalable, private payments," in *Proceedings of the Workshop on Privacy in the Electronic Society (WPES)*, 2023, pp. 1–16.

[5] C. LeMahieu, "Nano: A feeless distributed cryptocurrency network," White paper, 2018. [Online]. Available: https://content.nano.org/whitepaper/Nano_Whitepaper_en.pdf

[6] Mysten Labs, "Sui documentation: Epochs and reconfiguration," https://docs.sui.io/develop/sui-architecture/epochs, 2026, accessed Nov. 2026.

[7] K. Babel, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, A. Koshy, A. Sonnino, and M. Tian, "Mysticeti: Reaching the latency limits with uncertified DAGs," in *Network and Distributed System Security Symposium (NDSS)*, 2025.

[8] C. Cachin, K. Kursawe, F. Petzold, and V. Shoup, "Secure and efficient asynchronous broadcast protocols," in *Advances in Cryptology – CRYPTO 2001*, ser. LNCS, vol. 2139. Springer, 2001, pp. 524–541.

[9] L. Yang, Y. Gilad, and M. Alizadeh, "Practical rateless set reconciliation," in *Proceedings of the ACM SIGCOMM 2024 Conference*, 2024, pp. 595–612.

[10] V. Shoup, J. Sliwinski, and Y. Vonlanthen, "Kudzu: Fast and simple high-throughput BFT," in *39th International Symposium on Distributed Computing (DISC 2025)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 356. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2025, pp. 42:1–42:19.

[11] RsNano contributors, "Rsnano software repository," Software repository, base of the RAI prototype and of the evaluated baseline. [Online]. Available: https://github.com/rsnano-node/rsnano-node

[12] Mysten Labs, "Sui documentation: Transaction lifecycle," https://docs.sui.io/develop/transactions/transaction-lifecycle, 2026, accessed Nov. 2026.

[13] S. Blackshear, A. Chursin, G. Danezis, A. Kichidis, L. Kokoris-Kogias, X. Li, M. Logan, A. Menon, T. Nowacki, A. Sonnino, B. Williams, and L. Zhang, "Sui lutris: A blockchain combining broadcast and consensus," in *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2024, pp. 2606–2620.

[14] M. Camaioni, R. Guerraoui, J. Komatovic, M. Monti, P.-L. Roman, M. Vidigueira, and G. Voron, "Carbon: Scaling trusted payments with untrusted machines," *IEEE Transactions on Dependable and Secure Computing*, 2024, doi: 10.1109/TDSC.2024.3428617.

[15] Linera, "Linera developer documentation: Microchains and glossary," https://linera.dev/developers/core_concepts/microchains.html, 2026, accessed Nov. 2026.

[16] ——, "The linera-execution crate: AdminOperation and SystemOperation," https://docs.rs/linera-execution/latest/linera_execution/system/enum.AdminOperation.html, 2026, accessed Nov. 2026.

[17] L. Shangguan, "Transaction-driven dynamic reconfiguration for certificate-based payment systems," arXiv:2601.09146, 2026.

[18] L. Kokoris-Kogias, A. Sonnino, and G. Danezis, "Cuttlefish: Expressive fast path blockchains with FastUnlock," arXiv:2309.12715, 2023.

[19] R. Guerraoui, J. Komatovic, P. Kuznetsov, Y.-A. Pignolet, D.-A. Seredinschi, and A. Tonkikh, "Dynamic Byzantine reliable broadcast," in *24th International Conference on Principles of Distributed Systems (OPODIS 2020)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 184, 2021, pp. 23:1–23:18.

[20] P. Kuznetsov, Y.-A. Pignolet, P. Ponomarev, and A. Tonkikh, "Permissionless and asynchronous asset transfer," in *35th International Symposium on Distributed Computing (DISC 2021)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 209. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2021, pp. 28:1–28:19.

[21] P. Kuznetsov and A. Tonkikh, "Asynchronous reconfiguration with byzantine failures," *Distributed Computing*, vol. 35, no. 6, pp. 477–502, 2022.

[22] M. K. Aguilera, I. Keidar, D. Malkhi, and A. Shraer, "Dynamic atomic storage without consensus," *Journal of the ACM*, vol. 58, no. 2, 2011, article 7.

[23] E. Alchieri, A. Bessani, F. Greve, and J. da Silva Fraga, "Efficient and modular consensus-free reconfiguration for fault-tolerant storage," in *21st International Conference on Principles of Distributed Systems (OPODIS 2017)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 95, 2018, pp. 26:1–26:17.

[24] L. Jehl, R. Vitenberg, and H. Meling, "SmartMerge: A new approach to reconfiguration for atomic storage," in *Distributed Computing (DISC 2015)*, ser. LNCS, vol. 9363. Springer, 2015, doi: 10.1007/978-3-662-48653-5_11.

[25] P. Kuznetsov, T. Rieutord, and S. Tucci-Piergiovanni, "Reconfigurable lattice agreement and applications," in *23rd International Conference on Principles of Distributed Systems (OPODIS 2019)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 153, 2020, pp. 31:1–31:17.

[26] L. Lamport, D. Malkhi, and L. Zhou, "Reconfiguring a state machine," *SIGACT News*, vol. 41, no. 1, pp. 63–73, 2010.

[27] ——, "Stoppable Paxos," Microsoft Research, Tech. Rep., Apr. 2008.

[28] ——, "Vertical Paxos and primary-backup replication," in *28th ACM Symposium on Principles of Distributed Computing (PODC)*, 2009, pp. 312–313.

[29] D. Ongaro and J. Ousterhout, "In search of an understandable consensus algorithm," in *USENIX Annual Technical Conference (ATC)*, 2014, pp. 305–319.

[30] A. Bessani, J. Sousa, and E. E. P. Alchieri, "State machine replication for the masses with BFT-SMaRt," in *44th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2014.

[31] T.-H. H. Chan, R. Pass, and E. Shi, "PaLa: A simple partially synchronous blockchain," Cryptology ePrint Archive, Paper 2018/981, 2018.

[32] G. G. Gueta, I. Abraham, S. Grossman, D. Malkhi, B. Pinkas, M. K. Reiter, D.-A. Seredinschi, O. Tamir, and A. Tomescu, "SBFT: A scalable and decentralized trust infrastructure," in *49th Annual IEEE/IFIP International Conference on Dependable Systems and Networks (DSN)*, 2019, pp. 568–580.

[33] V. Shoup, "Sing a song of simplex," in *38th International Symposium on Distributed Computing (DISC 2024)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), vol. 319. Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2024, pp. 37:1–37:22.

[34] M. T. Goodrich and M. Mitzenmacher, "Invertible bloom lookup tables," in *49th Annual Allerton Conference on Communication, Control, and Computing*, 2011, pp. 792–799.

[35] D. Eppstein, M. T. Goodrich, F. Uyeda, and G. Varghese, "What's the difference? efficient set reconciliation without prior context," in *Proceedings of the ACM SIGCOMM 2011 Conference*, 2011, pp. 218–229.

[36] L. Lamport, *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers*. Addison-Wesley, 2002.